PROPUESTA DE PROYECTO

“Sistema Web Inteligente para la Gestión Integral de Servicios Universitarios”

1. Idea general

Los estudiantes desarrollarán una aplicación web moderna full-stack que permita a una universidad gestionar solicitudes de servicios realizadas por estudiantes, docentes y personal administrativo.

Por ejemplo, el sistema podría gestionar:

- Solicitudes de mantenimiento;
- Incidencias de infraestructura;
- Soporte informático;
- Solicitudes administrativas;
- Reservas de ambientes;
- Seguimiento de requerimientos;
- Asignación de responsables;
- Estados y prioridades;
- Notificaciones;
- Indicadores de atención.

El proyecto es suficientemente amplio para que puedan demostrar construcción real de software web, pero tiene que ser viable en 16 semanas.

2. Problema que deben resolver

Actualmente, muchas solicitudes universitarias pueden gestionarse mediante correos, formularios, llamadas telefónicas o sistemas independientes.

Esto genera problemas como:

- Pérdida o duplicación de solicitudes;
- Falta de seguimiento;
- Ausencia de trazabilidad;
- Dificultad para conocer quién es responsable;
- Poca información sobre tiempos de atención;
- Dificultad para medir la calidad del servicio;
- Falta de indicadores para la toma de decisiones.

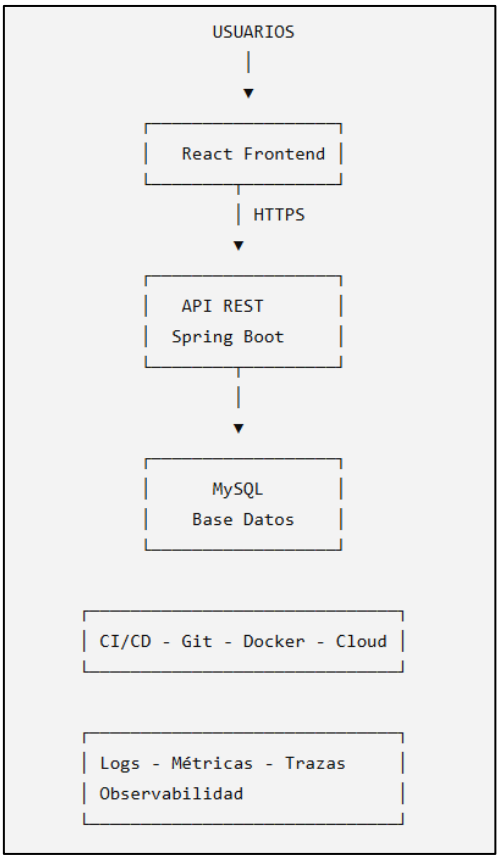
construir una plataforma web que permita registrar, gestionar, priorizar, atender y monitorear solicitudes de servicios universitarios, garantizando usabilidad, seguridad, calidad, rendimiento y disponibilidad

3. Producto final esperado

Los grupos deberán entregar una aplicación web full-stack funcional, accesible desde Internet o una infraestructura de demostración.

Una arquitectura posible sería:

PROPUESTA DE PROYECTO



4. Funcionalidades mínimas

El proyecto debera contener, como mínimo, los siguientes módulos.

Módulo 1. Autenticación

- Inicio de sesión.
- Registro de usuarios.
- Autenticación mediante JWT o sesión.
- Recuperación o cambio de contraseña, si el grupo tiene capacidad.
- Gestión de roles.

Roles sugeridos:

Rol	**Funciones**
Estudiante	Registrar consultas y solicitudes
Técnico	Atender solicitudes asignadas
Supervisor	Asignar, priorizar y supervisar
Administrador	Gestionar usuarios, categorías y configuración

Módulo 2. Registro de solicitudes

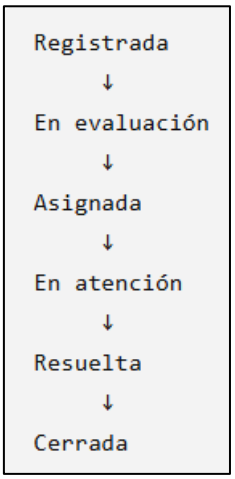
Cada solicitud debería contener:

- Código;
- Usuario;

- Categoría;
- Descripción;
- Ubicación;
- Prioridad;
- Fecha;
- Estado;
- Responsable;
- Evidencias;
- Comentarios.

Módulo 3. Gestión de solicitudes

Estados:



Módulo 4. Dashboard

Indicadores como:

- Solicitudes registradas;
- Solicitudes pendientes;
- Solicitudes atendidas;
- Solicitudes por categoría;
- Solicitudes por prioridad;
- Tiempo promedio de atención;
- Porcentaje de solicitudes resueltas;
- Solicitudes vencidas;
- Solicitudes por responsable.

Módulo 5. Administración

- Usuarios;
- Roles;
- Categorías;
- Prioridades;
- Estados;
- Áreas responsables.

PROPUESTA DE PROYECTO

5. Historias de usuario

El proyecto debe desarrollarse utilizando Historias de Usuario, épicas, tareas y especificaciones.

Ejemplo:

Épica

Gestión de solicitudes

Historia de usuario

Como estudiante, quiero registrar una solicitud de mantenimiento para poder informar un problema existente en mi facultad.

Criterios de aceptación

Escenario 1 – Registro correcto

```
Dado que el estudiante está autenticado
Cuando registra una solicitud con todos los datos obligatorios
Entonces el sistema debe registrar la solicitud
Y generar un código único
Y mostrar el estado "Registrada".
```

Escenario 2 – Datos incompletos

```
Dado que el estudiante está registrando una solicitud
Cuando omite un campo obligatorio
Entonces el sistema debe mostrar un mensaje de validación
Y no debe registrar la solicitud.
```

6. Plan de trabajo hasta final del ciclo

FASE I – Análisis, especificación y prototipo

Objetivo: definir qué se construirá y diseñar la primera versión de la solución.

Planteamiento

- Identificación del problema.
- Usuarios y actores.
- Alcance.
- Objetivos.
- Requerimientos iniciales.
- Arquitectura preliminar.
- Uso responsable de IA durante el desarrollo.

PROPUESTA DE PROYECTO

Producto:

- Documento del proyecto;
- Problema;
- Objetivos;
- Actores;
- Alcance;
- Arquitectura preliminar.

Especificaciones

- Épicas.
- Historias de usuario.
- División de historias.
- Tareas.
- Criterios de aceptación.
- Specs.
- Escenarios BDD.

UI/UX

- Wireframes.
- Mockups.
- Prototipo.
- HTML DOM.
- CSS.
- Diseño responsive.
- Usabilidad y UX.

JavaScript

- JavaScript moderno.
- Formularios.
- Validaciones.
- Eventos.
- Manipulación del DOM.
- Fetch.
- Primera interacción con APIs simuladas.

Los estudiantes presentan:

1. Problema.
2. Justificación.
3. Usuarios.
4. Alcance.
5. Requerimientos.
6. Épicas.
7. Historias de usuario.
8. Criterios de aceptación.
9. Arquitectura preliminar.
10. Wireframes/mockups.

11. Prototipo.
12. Evidencia del primer desarrollo.

Producto: – Diseño y especificación.

FASE II – Construcción Full Stack

React

Construcción del frontend:

- Componentes;
- Props;
- State;
- Eventos;
- Listas;
- Renderizado condicional;
- Componentes reutilizables.

Backend

Construcción del backend con:

- Spring Boot;
- Controladores;
- Servicios;
- DTO;
- Validaciones;
- Arquitectura por capas;
- Conexión MySQL.

Persistencia e integración

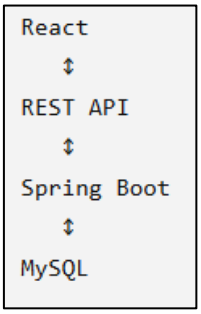
Implementación de:

- CRUD;
- Relaciones;
- GET;
- POST;
- PUT;
- DELETE;
- Fetch;
- Manejo de estado;
- Cookies;
- LocalStorage/SessionStorage.

Primera versión funcional

PROPUESTA DE PROYECTO

Integración:



Debe demostrarse una primera versión funcional end-to-end.

Por ejemplo:

Login → registrar solicitud → guardar en MySQL → consultar solicitud → modificar estado.

Producto: – MVP funcional.

Semana 10 – presentación parcial (subida de informe al classroom)

FASE III – APIs, nube, DevOps y seguridad

Cloud + CI/CD

- Selección de plataforma cloud.
- Docker.
- Ambientes de desarrollo/prueba/producción.
- Git.
- Ramas.
- Integración continua.
- Despliegue.
- Pipeline CI/CD.

API REST

- Diseño de recursos.
- Rutas.
- Parámetros.
- códigos HTTP;
- documentación;
- pruebas de API;
- panorama de GraphQL/gRPC/webhooks;
- API Gateway.

Seguridad

Implementar:

- Autenticación;

- Autorización;
- Roles;
- JWT o sesión;
- Validación de entradas;
- Protección de secretos;
- Variables de entorno;
- Prevención de hardcode;
- Principios OWASP.

Testing

- Pruebas unitarias;
- Pruebas de integración;
- Automatización;
- Criterios de aceptación;
- Pruebas de API;
- Generación asistida por IA;
- Revisión de pruebas generadas por IA.

Debe demostrarse:

- Aplicación desplegada;
- API documentada;
- CI/CD;
- Autenticación;
- Autorización;
- Seguridad;
- Pruebas;
- Evidencias de automatización.

Producto: – Release Candidate.

FASE IV – Calidad, performance, observabilidad y entrega

Pruebas funcionales y performance

Realizar:

- Pruebas funcionales;
- Pruebas de aceptación;
- Pruebas de usuario;
- Pruebas de seguridad;
- SAST;
- DAST;
- Pruebas de carga;
- Pruebas de estrés.

Los estudiantes deben generar un informe de pruebas.

PROPUESTA DE PROYECTO

Observabilidad

Implementar:

- Logs;
- Métricas;
- Trazas;
- Monitoreo;
- Alertas;
- Identificación de incidentes;
- Depuración.

Se puede introducir OpenTelemetry como parte de la observabilidad.

Calidad y revisión final

Realizar:

- Revisión de arquitectura;
- Revisión de código;
- Patrones de diseño;
- Eliminación de hardcode;
- Gestión de secretos;
- Manejo de errores;
- Documentación;
- Revisión de escenarios;
- Revisión del uso de IA.

Release final

Preparación de:

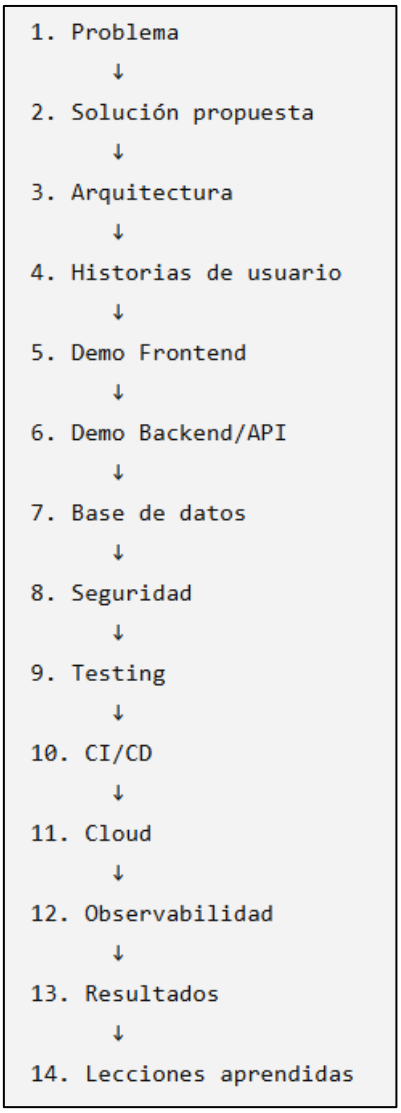
- Versión definitiva;
- Documentación;
- Manual de usuario;
- Manual técnico;
- Repositorio;
- Evidencias;
- Presentación;
- Demostración.

Semana 16 - SUSTENTACIÓN FINAL (subida de informe final al classroom y exposición)

Cada grupo debe realizar una demostración integral del sistema, no solamente una presentación PowerPoint.

La sustentación podría seguir esta secuencia:

PROPUESTA DE PROYECTO



7. Elemento diferenciador: uso de IA

Dado que el proyecto incorpore desarrollo asistido con IA, Copilot, generación de pruebas con IA, seguridad al utilizar IA y revisión de escenarios mediante IA , incluiría un requisito transversal:

Cada grupo deberá mantener un "Registro de uso de IA"

Por cada utilización significativa de IA deberán indicar:

Campo	**Ejemplo**
Herramienta	Copilot
Prompt	Generar prueba unitaria para…
Resultado generado	Código
¿se utilizó?	Parcialmente
Validación realizada	Pruebas manuales
Modificaciones	3
Responsable	Integrante 2

Esto permite evaluar que la IA sea utilizada como herramienta de ingeniería y no como sustituto del aprendizaje.

8. Requisito importante para evitar proyectos superficiales

Se establece como regla que no se calificará solamente que “la aplicación funcione”.

Para obtener una buena calificación, el grupo debe demostrar:

Especificar → Diseñar → Construir → Integrar → Probar → Asegurar → Desplegar → Monitorear → Mejorar

De esta manera, el proyecto realmente integra prácticamente todo el contenido del curso y no se convierte simplemente en "hacer una página web con React".

Resultado final esperado

Al finalizar el ciclo, cada grupo debería poder afirmar:

“Hemos construido, probado, asegurado, desplegado y monitorizado una aplicación web full-stack utilizando especificaciones, React, JavaScript, Spring Boot, MySQL, APIs REST, autenticación, Docker, CI/CD, pruebas automatizadas, prácticas OWASP y observabilidad.”

Tener en cuenta que el nivel del proyecto no debería centrarse únicamente en programación, sino en demostrar el ciclo completo de construcción de un producto de software web profesional.